

File systems

From [Wikipedia](#):

In computing, a file system or filesystem controls how data is stored and retrieved. Without a file system, information placed in a storage medium would be one large body of data with no way to tell where one piece of information stops and the next begins. By separating the data into pieces and giving each piece a name, the information is easily isolated and identified. Taking its name from the way paper-based information systems are named, each group of data is called a "file". The structure and logic rules used to manage the groups of information and their names is called a "file system".

Individual drive partitions can be set up using one of the many different available file systems. Each has its own advantages, disadvantages, and unique idiosyncrasies. A brief overview of supported filesystems follows; the links are to Wikipedia pages that provide much more information.

Contents [hide]

1

Types of file systems

1.1

Journaling

1.2

FUSE-based file systems

1.3

Stackable file systems

1.4

Read-only file systems

1.5

Clustered file systems

1.6

Shared-disk file system

2

Identify existing file systems

3

Create a file system

4

Mount a file system

4.1

List mounted file systems

4.2

Unmount a file system

5

See also

- Related articles
- Partitioning

Device file#lsblk

File permissions and attributes

fsck

fstab

List of applications#Mount tools

QEMU#Mounting a partition from a raw image

udev

udisks

umask

USB storage devices

Types of file systems



The factual accuracy of this article or section is disputed.

Reason:

`/proc/filesystems` only lists file systems whose modules are either built-in or currently loaded. Since [Arch kernels](#) have most of the file systems built as loadable modules, `/proc/filesystems` will show very few, if any, usable file systems. (Discuss in [Talk:File systems](#))



See [filesystems\(5\)](#) for a general overview and [Wikipedia:Comparison of file systems](#) for a detailed feature comparison. File systems supported by the kernel are listed in `/proc/filesystems`.

In-tree and FUSE file systems					
File system ↕	Creation command ↕	Userspace utilities ↕	Archiso [1] ↕	Kernel documentation [2] ↕	Notes ↕
Btrfs	mkfs.btrfs (8)	btrfs-progs	Yes	btrfs.html ↕	Stability status ↕
VFAT	mkfs.fat (8)	dosfstools	Yes	vfat.html ↕	Windows 9x ↕ file system
exFAT ↕	mkfs.exfat (8)	exfatprogs	Yes		Native file system in Linux 5.4. [3] ↕
	mkexfatfs (8)	exfat-utils	No	N/A (FUSE-based)	
F2FS	mkfs.f2fs (8)	f2fs-tools	Yes	f2fs.html ↕	Flash-based devices
ext3	mkfs.ext3 (8)	e2fsprogs	Yes	ext3.html ↕	
ext4	mkfs.ext4 (8)	e2fsprogs	Yes	ext4.html ↕	
HFS ↕	mkfs.hfsplus (8)	hfsprogs ^{AUR}	No	hfs.html ↕	Classic Mac OS ↕ file system
HFS+ ↕	mkfs.hfsplus (8)	hfsprogs ^{AUR}	No	hfsplus.html ↕	macOS ↕ (8–10.12) file system
JFS	mkfs.jfs (8)	jfsutils	Yes	jfs.html ↕	
NILFS2 ↕	mkfs.nilfs2 (8)	nilfs-utils	Yes	nilfs2.html ↕	Raw flash devices, e.g. SD card
NTFS			No	ntfs3.html ↕	Windows NT ↕ file system. New driver ↕, available since Linux 5.15.
			No	ntfs.html ↕	Old driver. Has very limited write support. Officially supported kernels are built without <code>CONFIG_NTFS_FS</code> so this driver is not available.
	mkfs.ntfs (8)	ntfs-3g [4] ↕	Yes	N/A (FUSE-based)	FUSE driver with extended capabilities.
ReiserFS ↕	mkfs.reiserfs (8)	reiserfsprogs	Yes		
UDF ↕	mkfs.udf (8)	udftools	Yes	udf.html ↕	
XFS	mkfs.xfs (8)	xfsprogs	Yes	xfs.html ↕ xfs-delayed-logging-design.html ↕ xfs-self-describing-metadata.html ↕	

Out-of-tree file systems				
File system ↕	Creation command ↕	Kernel patchset ↕	Userspace utilities ↕	Notes ↕
APFS ↕	mkapfs (8)	linux-apfs-rw-dkms-git ^{AUR}	apfsprogs-git ^{AUR}	macOS ↕ (10.13 and newer) file system. Read only, experimental.
Bcachefs	bcachefs (8)	linux-bcachefs-git ^{AUR}	bcachefs-tools-git ^{AUR}	
Reiser4	mkfs.reiser4 (8)		reiser4progs ^{AUR}	
ZFS		zfs-linux ^{AUR} , zfs-dkms ^{AUR}	zfs-utils ^{AUR}	OpenZFS ↕ port

Journaling

All the above file systems with the exception of exFAT, ext2, FAT16/32, Reiser4 (optional), Btrfs and ZFS, use [journaling](#) ↕. Journaling provides fault-resilience by logging changes before they are committed to the file system. In the event of a system crash or power failure, such file systems are faster to bring back online and less likely to become corrupted. The logging takes place in a dedicated area of the file system.

Not all journaling techniques are the same. Ext3 and ext4 offer data-mode journaling, which logs both data and meta-data, as well as possibility to journal only meta-data changes. Data-mode journaling comes with a speed penalty and is not enabled by default. In the same vein, [Reiser4](#) offers so-called ["transaction models"](#) ↕ which not only change the features it provides, but in its journaling mode. It uses a different journaling techniques: a special model called [wandering logs](#) ↕ which eliminates the need to write to the disk twice, [write-anywhere](#)—a pure copy-on-write approach (mostly equivalent to btrfs' default but with a fundamentally different "tree" design) and a combined approach called [hybrid](#) which heuristically alternates between the two former.

Note: Reiser4 does provide an almost equivalent to ext4's default journaling behavior (meta-data only) with the use of the **node41** plugin which also features meta-data and inline checksums, optionally combined with the wandering logs behaviour it provides depending on what transaction model is chosen at mount time.

The other file systems provide ordered-mode journaling, which only logs meta-data. While all journaling will return a file system to a valid state after a crash, data-mode journaling offers the greatest protection against corruption and data loss. There is a compromise in system performance, however, because data-mode journaling does two write operations: first to the journal and then to the disk (which Reiser4 avoids with its "wandering logs" feature). The trade-off between system speed and data safety should be considered when choosing the file system type. Reiser4 is the only file system that by design operates on full atomicity and also provides checksums for both meta-data and inline data (operations entirely occur, or they entirely do not and does not corrupt or destroy data due to operations half-occurring) and by design is therefore much less prone to data loss than other file systems like **Btrfs**.

File systems based on copy-on-write (also known as write-anywhere), such as Reiser4, Btrfs and ZFS, have no need to use traditional journal to protect metadata, because they are never updated in-place. Although Btrfs still has a journal-like log tree, it is only used to speed-up fdatasync/fsync.

FUSE-based file systems

See **FUSE**.

Stackable file systems

- aufs** — Advanced multi-layered unification file system, a FUSE based union file system, a complete rewrite of Unionfs, was rejected from Linux mainline and instead OverlayFS was merged into the Linux Kernel.
<http://aufs.sourceforge.net> || [linux-aufs](#)^{AUR}
- eCryptfs** — The enterprise cryptographic file system is a package of disk encryption software for Linux. It is implemented as a POSIX-compliant file system-level encryption layer, aiming to offer functionality similar to that of GnuPG at the operating system level.
<https://ecryptfs.org> || [ecryptfs-utils](#)
- mergerfs** — a FUSE based union file system.
<https://github.com/trapexit/mergerfs> || [mergerfs](#)^{AUR}
- mhddfs** — Multi-HDD FUSE file system, a FUSE based union file system.
<http://mhddfs.uvw.ru> || [mhddfs](#)^{AUR}
- overlayfs** — OverlayFS is a file system service for Linux which implements a union mount for other file systems.
<https://www.kernel.org/doc/html/latest/filesystems/overlayfs.html> || [linux](#)
- Unionfs** — Unionfs is a file system service for Linux, FreeBSD and NetBSD which implements a union mount for other file systems.
<https://unionfs.filesystems.org/> || not packaged? [search in AUR](#)
- unionfs-fuse** — A user space Unionfs implementation.
<https://github.com/rpodgorny/unionfs-fuse> || [unionfs-fuse](#)

Read-only file systems

- EROFS** — Enhanced Read-Only File System is a lightweight read-only file system, it aims to improve performance and compress storage capacity.
<https://www.kernel.org/doc/html/latest/filesystems/erofs.html> || [erofs-utils](#)
- SquashFS** — SquashFS is a compressed read only file system. SquashFS compresses files, inodes and directories, and supports block sizes up to 1 MB for greater compression.
<https://github.com/plougher/squashfs-tools> || [squashfs-tools](#)

Clustered file systems

- Ceph** — Unified, distributed storage system designed for excellent performance, reliability and scalability.
<https://ceph.com/> || [ceph](#)
- Glusterfs** — Cluster file system capable of scaling to several peta-bytes.
<https://www.gluster.org/> || [glusterfs](#)
- IPFS** — A peer-to-peer hypermedia protocol to make the web faster, safer, and more open. IPFS aims replace HTTP and build a better web for all of us. Uses blocks to store parts of a file, each network node stores only content it is interested, provides deduplication, distribution, scalable system limited only by users. (currently in alpha)
<https://ipfs.io/> || [go-ipfs](#)
- MinIO** — MinIO offers high-performance, S3 compatible object storage.
<https://min.io> || [minio](#)
- MooseFS** — MooseFS is a fault tolerant, highly available and high performance scale-out network distributed file system.
<https://moosefs.com> || [moosefs](#)
- OpenAFS** — Open source implementation of the AFS distributed file system
<https://www.openafs.org> || [openafs](#)^{AUR}
- OrangeFS** — OrangeFS is a scale-out network file system designed for transparently accessing multi-server-based disk storage, in parallel. Has optimized MPI-IO support for parallel and distributed applications. Simplifies the use of parallel storage not only for Linux clients, but also for Windows, Hadoop, and WebDAV. POSIX-compatible. Part of Linux kernel since version 4.6.
<https://www.orangefs.org/> || not packaged? [search in AUR](#)
- Sheepdog** — Distributed object storage system for volume and container services and manages the disks and nodes intelligently.
<https://sheepdog.github.io/sheepdog/> || [sheepdog](#)^{AUR}
- Tahoe-LAFS** — Tahoe Least-Authority File System is a free and open, secure, decentralized, fault-tolerant, peer-to-peer distributed data store and distributed file system.
<https://tahoe-lafs.org/> || [tahoe-lafs](#)^{AUR}

Shared-disk file system

- GFS2** — GFS2 allows all members of a cluster to have direct concurrent access to the same shared block storage
<https://pagure.io/gfs2-utils> || [gfs2-utils](#)^{AUR}
- OCFS2** — The Oracle Cluster File System (version 2) is a shared disk file system developed by Oracle Corporation and released under the GNU General Public License
<https://oss.oracle.com/projects/ocfs2/> || [ocfs2-tools](#)^{AUR}
- VMware VMFS** — VMware's VMFS (Virtual Machine File System) is used by the company's flagship server virtualization suite, vSphere.
<https://www.vmware.com/products/vi/esx/vmfs.html> || [vmfs-tools](#)^{AUR}

Identify existing file systems

To identify existing file systems, you can use **lsblk**:

\$ lsblk -f				
NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
sdb				
└─sdb1	vfat	Transcend	4A3C-A9E9	

An existing file system, if present, will be shown in the **FSTYPE** column. If **mounted**, it will appear in the **MOUNTPOINT** column.

Create a file system

File systems are usually created on a **partition**, inside logical containers such as **LVM**, **RAID** and **dm-crypt**, or on a regular file (see [Wikipedia:Loop device](#)). This section describes the partition case.

Note: File systems can be written directly to a disk, known as a *superfloppy* or **partitionless disk**. Certain limitations are involved with this method, particularly if **booting** from such a drive. See [Btrfs#Partitionless Btrfs disk](#) for an example.

Warning:

- After creating a new file system, data previously stored on this partition can unlikely be recovered. **Create a backup of any data you want to keep.**
- The purpose of a given partition may restrict the choice of file system. For example, an **EFI system partition** must contain a **FAT32** file system, and the file system containing the `/boot` directory must be supported by the **boot loader**.

Before continuing, **identify the device** where the file system will be created and whether or not it is mounted. For example:

```
$ lsblk -f
```

NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
sda				
_sda1			C4DA-2C4D	
_sda2	ext4		5b1564b2-2e2c-452c-bcfa-d1f572ae99f2	/mnt
_sda3			56adc99b-a61e-46af-aab7-a6d07e504652	

Mounted file systems **must** be **unmounted** before proceeding. In the above example an existing file system is on `/dev/sda2` and is mounted at `/mnt`. It would be unmounted with:

```
# umount /dev/sda2
```

To find just mounted file systems, see [#List mounted file systems](#).

To create a new file system, use [mkfs\(8\)](#). See [#Types of file systems](#) for the exact type, as well as userspace utilities you may wish to install for a particular file system.

For example, to create a new file system of type **ext4** (common for Linux data partitions) on `/dev/sda1`, run:

```
# mkfs.ext4 /dev/sda1
```

Tip:

- Use the `-L` flag of *mkfs.ext4* to specify a **file system label**. *e2label* can be used to change the label on an existing file system.
- File systems may be *resized* after creation, with certain limitations. For example, an **XFS** file system's size can be increased, but it cannot reduced. See [Wikipedia:Comparison of file systems#Resize capabilities](#) and the respective file system documentation for details.

The new file system can now be mounted to a directory of choice.

Mount a file system

To manually mount a file system located on a device (e.g., a partition) to a directory, use [mount\(8\)](#). This example mounts `/dev/sda1` to `/mnt`.

```
# mount /dev/sda1 /mnt
```

This attaches the file system on `/dev/sda1` at the directory `/mnt`, making the contents of the file system visible. Any data that existed at `/mnt` before this action is made invisible until the device is unmounted.

fstab contains information on how devices should be automatically mounted if present. See the [fstab](#) article for more information on how to modify this behavior.

If a device is specified in `/etc/fstab` and only the device or mount point is given on the command line, that information will be used in mounting. For example, if `/etc/fstab` contains a line indicating that `/dev/sda1` should be mounted to `/mnt`, then the following will automatically mount the device to that location:

```
# mount /dev/sda1
```

Or

```
# mount /mnt
```

mount contains several options, many of which depend on the file system specified. The options can be changed, either by:

- using flags on the command line with *mount*
- editing [fstab](#)
- creating [udev](#) rules
- [compiling the kernel yourself](#)
- or using file system-specific mount scripts (located at `/usr/bin/mount.*`).

See these related articles and the article of the file system of interest for more information.

Tip:

- File systems can also be mounted with *systemd-mount* instead of *mount*. If the mount point is not specified, the file system will be mounted at `/run/media/system/device_identifier/`. This allows to easily mount a file system without having to decide where to mount it. See [systemd-mount\(1\)](#) for usage and more details.
- To mount file systems as an ordinary user, see [udisks#Usage](#). This also allows mounting without having root permissions, a full graphical environment or a file manager which utilizes udisks.

List mounted file systems

To list all mounted file systems, use [findmnt\(8\)](#):

```
$ findmnt
```

findmnt takes a variety of arguments which can filter the output and show additional information. For example, it can take a device or mount point as an argument to show only information on what is specified:

```
$ findmnt /dev/sda1
```

findmnt gathers information from `/etc/fstab`, `/etc/mtab`, and `/proc/self/mounts`.

Unmount a file system

To unmount a file system use [umount\(8\)](#). Either the device containing the file system (e.g., `/dev/sda1`) or the mount point (e.g., `/mnt`) can be specified:

```
# umount /dev/sda1
```

or

```
# umount /mnt
```

See also

- [filesystems \(5\)](#)
- [systemd-mount \(1\)](#)
- [Documentation of file systems supported by Linux](#) 
- [Wikipedia:File systems](#) 
- [Wikipedia:Mount \(Unix\)](#) 

Categories: [File systems](#) | [Lists](#)

This page was last edited on 21 November 2021, at 06:18.

Content is available under [GNU Free Documentation License 1.3 or later](#) unless otherwise noted.

[Privacy policy](#) [About ArchWiki](#) [Disclaimers](#)

FAT



Related articles

File systems

From [Wikipedia:File Allocation Table](#):

File Allocation Table (FAT) is a computer file system architecture and a family of industry-standard file systems utilizing it. The FAT file system is a legacy file system which is simple and robust. It offers good performance even in light-weight implementations, but cannot deliver the same performance, reliability and scalability as some modern file systems. It is, however, supported for compatibility reasons by nearly all currently developed operating systems for personal computers and many mobile devices and embedded systems, and thus is a well-suited format for data exchange between computers and devices of almost any type and age from 1981 up to the present.

Contents [hide]

- 1 File system creation
- 2 Kernel configuration
- 3 Writing to FAT32 as normal user
- 4 Detecting FAT type
- 5 See also

File system creation

To create a FAT filesystem, [install dosfstools](#).

`mkfs.fat` supports creating FAT12, FAT16 and FAT32, see [Wikipedia:File Allocation Table#Types](#) for an explanation on their differences. `mkfs.fat` will select the FAT type based on the partition size, to explicitly create a certain type of FAT filesystem use the `-F` option. See [mkfs.fat\(8\)](#) for more information.

Tip: For most situations you will want to use FAT32.

Format a partition to FAT32:

```
# mkfs.fat -F 32 /dev/partition
```

Note: `mkfs.vfat` and `mkfs.msdos` are both symlinks to `mkfs.fat`, they are the same utility.

Kernel configuration

Here is an example of the default *mount* configuration in the kernel:

```
$ zgrep -e FAT -e DOS /proc/config.gz | sort -r

# DOS/FAT/NT Filesystems
CONFIG_FAT_FS=m
CONFIG_MSDFS_PARTITION=y
CONFIG_FAT_FS=m
CONFIG_MSDFS_FS=m
CONFIG_VFAT_FS=m
CONFIG_FAT_DEFAULT_CODEPAGE=437
CONFIG_FAT_DEFAULT_IOCHARSET="iso8859-1"
CONFIG_NCPFS_SMALLDOS=y
```

A short description of the options:

- Language settings: `CONFIG_FAT_DEFAULT_CODEPAGE`, `CONFIG_FAT_DEFAULT_IOCHARSET`
- All filenames to lower letters on a FAT partitions if enabled: `CONFIG_NCPFS_SMALLDOS`
- Enables support of the FAT file systems: `CONFIG_FAT_FS`, `CONFIG_MSDFS_FS`, `CONFIG_VFAT_FS`
- Enables support of a FAT partitioned harddisks on 86x PCs: `CONFIG_MSDFS_PARTITION`

If the partition type detected by mount is VFAT then it will run the `/usr/bin/mount.vfat` script.

```
/usr/bin/mount.vfat

#!/bin/bash
#mount VFAT with full rw (read-write) permissions for all users
#/usr/bin/mount -i -t vfat -o umask=0000,iocharset=utf8 "$@"
#The above is the same as
mount -i -t vfat -o iocharset=utf8,umask=0000,dmask=0000 "$@"
```

Writing to FAT32 as normal user

To write on a FAT32 partition, you must make a few changes to the `fstab` file.

```
/etc/fstab

/dev/sdxY /mnt/some_folder vfat user,rw
```

The `user` option means that any user (even non-root) can mount and unmount the partition `/dev/sdx`. `rw` gives read-write access.

For example, if your FAT32 partition is on `/dev/sda9`, and you wish to mount it to `/mnt/fat32`, then you would use:

```
/etc/fstab

/dev/sda9 /mnt/fat32 vfat user,rw
```

Now, any user can mount it with:

```
$ mount /mnt/fat32
```

And unmount it with:

```
$ umount /mnt/fat32
```

Note that FAT does not support Linux file permissions. Each file will also appear to be executable. You may want to use the `showexec` option to only mark Windows executables (com, exe, bat) as executable. See [mount \(8\)](#) for more options.

Detecting FAT type

If you need to know which [type of FAT file system](#) a partition uses, use the `file` command:

```
# file -s /dev/partition
```

```
/dev/partition: DOS/MBR boot sector, code offset 0x3c+2, OEM-ID "mkfs.fat", sectors/cluster 4, root entries 512, sectors 4096 (volumes <=32 MB), Media descriptor 0xf8, sectors/FAT 3, sectors/track 32, heads 64, serial number 0x5bc09c21, unlabeled, FAT (12 bit)
```

Alternatively you can use `minfo` from the `mtools` package:

```
# minfo -i /dev/partition ::
```

```
device information:
=====
```

```
filename="/dev/partition"
sectors per track: 32
heads: 64
cylinders: 2
```

```
media byte: f8
```

```
mformat command line: mformat -t 2 -h 64 -s 32 -i "/dev/partition" ::
```

```
bootsector information
=====
```

```
banner:"mkfs.fat"
sector size: 512 bytes
cluster size: 4 sectors
reserved (boot) sectors: 1
fats: 2
max available root directory slots: 512
small size: 4096 sectors
media descriptor byte: 0xf8
sectors per fat: 3
sectors per track: 32
heads: 64
hidden sectors: 0
big size: 0 sectors
physical drive id: 0x80
reserved=0x0
dos4=0x29
serial number: 5BC09C21
disk label="NO NAME"
disk type="FAT12"
```

See also

Category: [File systems](#)

This page was last edited on 17 February 2021, at 15:58.

Content is available under [GNU Free Documentation License 1.3](#) or [later](#) unless otherwise noted.

[Privacy policy](#) [About ArchWiki](#) [Disclaimers](#)